



Sockets no bloqueantes

Socket options
Signals
Nonblocking i/o

Socket options

```
#include <sys/socket.h>
```

```
int getsockopt(int socket, int level,  
              int optionName,  
              void *restrict optionValue,  
              socklen_t *restrict optionLen);
```

```
int setsockopt(int socket, int level,  
              int optionName,  
              const void *optionValue,  
              socklen_t optionLen);
```

Socket options: ejemplo

```
/* Do I linger on close? */
int option_len = sizeof(struct linger);
struct linger ling;
int rc = getsockopt(
    s, SOL_SOCKET, SO_LINGER, (void *) &ling, &option_len);
if (rc == 0) {
    if (option_len == sizeof(ling)) {
        if (ling.l_onoff)
            /* yes I linger */
        else
            /* no I do not */
    }
}
```

Socket options

- Level
 - SOL_SOCKET: opción independiente del protocolo
 - IPPROTO_TCP: relativo a TCP
 - IPPROTO_IP: relativo al protocolo de red IP v4
 - IPPROTO_IPV6: relativo a IP v6

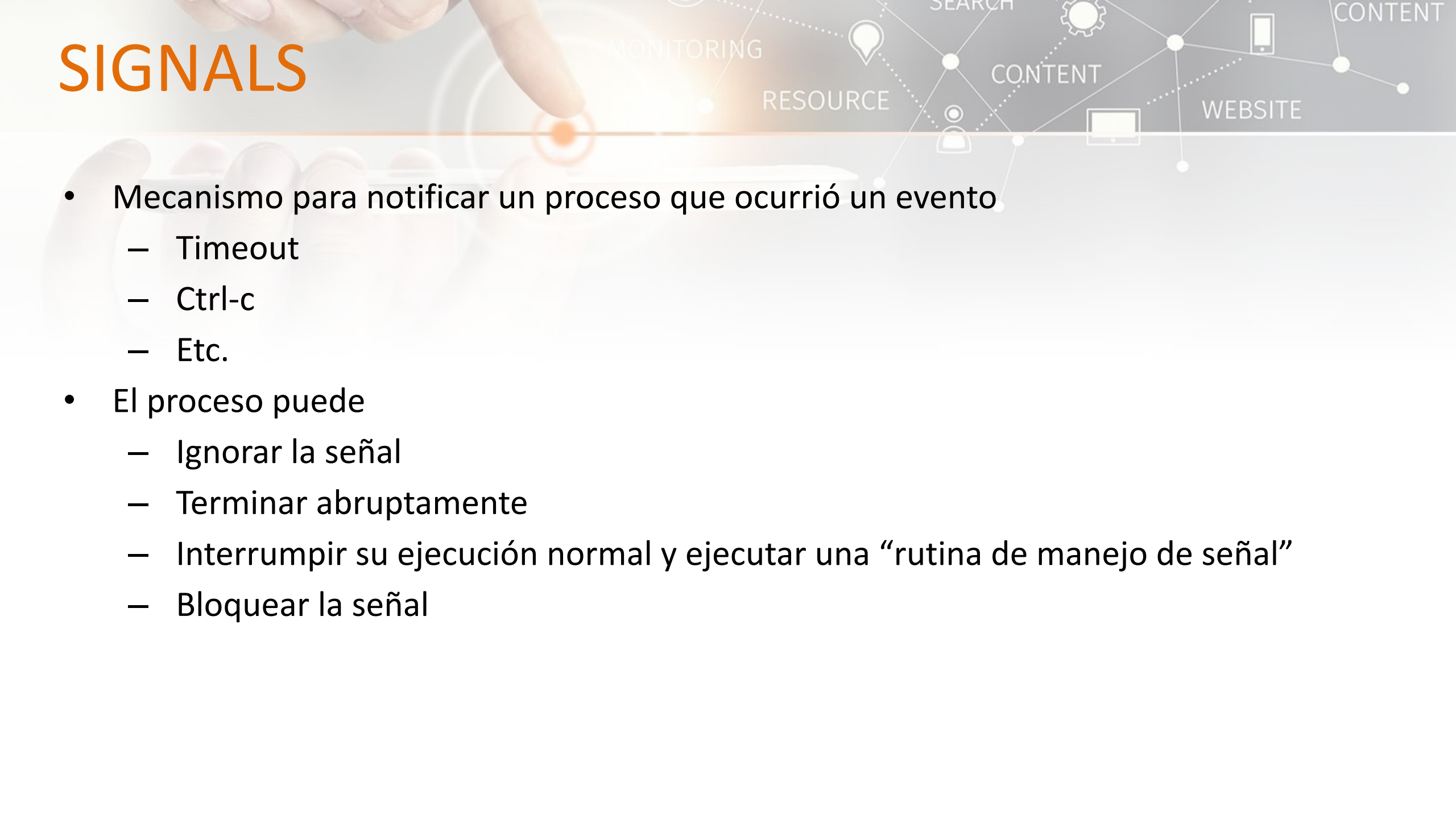
Socket options: listado incompleto

- SOL_SOCKET
 - SO_BROADCAST: 0 ó 1, si permite broadcast
 - SO_KEEPALIVE
 - SO_RCVBUF: tamaño en bytes del buffer de entrada
 - SO_RCVLOWAT: Mínimo tamaño en bytes para procesar el input.
 - SO_SNDLOWAT: ídem para la salida
 - SO_SNDBUF: tamaño del buffer de salida
 - SO_RCVTIMEO: máximo timeout para operaciones de lectura
 - SO_LINGER: 0 para enviar RST al hacer close()-> "*connection reset by peer*"
 - SO_REUSEADDR: en caso de abortar el programa, reusar dirección y Puerto
 - SO_REUSEPORT

Socket options: listado incompleto

- IPPROTO_TCP
 - TCP_KEEPIDLE, TCP_KEEPINTVL, TCP_KEEPCNT
 - TCP_NODELAY: 0,1
- IPPROTO_IP
 - IP_TTL: [0,255]
 - IP_MULTICAST_LOOP: habilita al socket multicast a recibir paquetes enviados por él.
 - IP_ADD_MEMBERSHIP
 - IP_DROP_MEMBERSHIP
- IPPROTO_IPV6
 - IPV6_V6ONLY

SIGNALS



- Mecanismo para notificar un proceso que ocurrió un evento
 - Timeout
 - Ctrl-c
 - Etc.
- El proceso puede
 - Ignorar la señal
 - Terminar abruptamente
 - Interrumpir su ejecución normal y ejecutar una “rutina de manejo de señal”
 - Bloquear la señal

SIGNALS

Señal	Evento	Acción por defecto
SIGALARM	Expiró un timer	Termina ejecución
SIGCHILD	“exit” de proceso hijo	Ignorar
SIGINT	Carácter interrupción (CTRL-C)	Terminar
SIGIO	Socket listo para I/O	Ignorar
SIGPIPE	Intento de escribir en socket cerrado	Terminar
SIGSEGV	“Segmentation fault”	Terminar
SIGTERM	Terminación “amable” (kill)	Terminar
SIGKILL	Terminación forzosa	Terminar

SIGNALS: manejar el comportamiento

```
int sigaction(int signum, const struct sigaction *act,  
              struct sigaction *oldact);
```

```
struct sigaction {  
    void      (*sa_handler) (int);  
    sigset_t  sa_mask;  
    int       sa_flags;  
};
```

SIG_IGN, SIG_DFL o función
que “capture” la señal

Señales a bloquear
mientras se ejecuta
sa_handler

SIGNALS

- `int sigemptyset(sigset_t * set);`
- `int sigfillset(sigset_t * set);`
- `int sigaddset(sigset_t * set, int signal);`
- `int sigdelset(sigset_t * set, int signal);`

```
sigset_t mask;
sigemptyset (&mask);
sigaddset (&mask, SIGTERM);
sigaddset (&mask, SIGINT);
struct sigaction handler = {SIG_IGN, &mask, 0};
if (sigaction(SIGINT, &handler, NULL) < 0) {
    perror (" failed trying to ignore SIGINT ");
    return 1;
}
```

SIGNALS: ejemplo

```
int main(int argc, char *argv[]) {
    struct sigaction handler; // Signal handler specification structure

    // Set InterruptSignalHandler() as handler function
    handler.sa_handler = InterruptSignalHandler;

    // Create mask that blocks all signals
    if (sigfillset(&handler.sa_mask) < 0)
        ERROR("sigfillset() failed");
    handler.sa_flags = 0; // No flags

    // Set signal handling for interrupt signal
    if (sigaction(SIGINT, &handler, 0) < 0)
        ERROR("sigaction() failed for SIGINT");
}
```

SIGNALS: ejemplo (cont.)

```
for (;;)
    pause(); // Suspend program until signal received

exit(0);
}

void InterruptSignalHandler(int signalType) {
    puts("Interrupt Received.  Exiting program.");
    exit(1);
}
```

SIGNALS: anidamiento

- ¿Qué sucede si se produce una señal y se está atendiendo otra?
- El manejo es pospuesto hasta que se termina de manejar la señal actual
- ¡Las señales no se encolan !
- Ejemplo:

```
void InterruptSignalHandler(int signalType) {  
    puts("Interrupt Received.\n");  
    sleep(3);  
}
```

SIGNALS y sockets

Si una señal es enviada mientras el programa está bloqueado en un “socket call” (`recv()`, `connect()`, `select()` etc.) y para esa señal se especificó un “handler”, entonces cuando el “handler” termina

- el “socket call” retorna -1
- `errno = EINTR`

```
while (1) {  
    errno=0;  
    int rc = recv (sock, buf, 1, 0);  
    if (rc == -1 && errno == EINTR)  
        continue;  
    ...  
}
```


Blocking y timeouts

- ◆ Llamadas a E/S pueden bloquear por diversas razones
 - ◆ `recv()` , `recvfrom()` : no hay datos en buffer de entrada
 - ◆ `send()` : no hay suficiente espacio en buffer de salida
 - ◆ `sendto()` : poco probable, pero puede bloquear
 - ◆ `connect()` , `accept()` : hasta que se completa la conexión

Blocking y timeouts

- ◆ ¿Y si un programa quiere hacer algo mientras espera por una conexión?
- ◆ ¿Qué sucede con datagramas UDP perdidos?

```
...
numBytes = recvfrom(sock, buffer, MAXSTRINGLENGTH, 0,
    (struct sockaddr *) &fromAddr, &fromAddrLen);
if (numBytes < 0)
    ERROR("recvfrom() failed");
...
```

Nonblocking i/o

- ◆ Para “romper” el bloqueo podemos
 - ◆ usar sockets no bloqueantes
 - ◆ I/O asincrónico
 - ◆ timeouts

Nonblocking i/o: sockets no bloqueantes

- ◆ Todas las llamadas a sockets hacerlas no bloqueantes
- ◆ Si la operación es inmediata retorna éxito
- ◆ Sinó, retorna error: se debe distinguir si es por error o recurso no disponible.
- ◆ Se cambia el bloqueo con `fcntl()`

```
#include <fcntl.h>
int s = socket(PF_INET, SOCK_STREAM, 0);

int flags = fcntl(s, F_GETFL, 0);
fcntl(s, F_SETFL, flags | O_NONBLOCK);
```

Nonblocking i/o: sockets no bloqueantes

¿Qué sucede si una función necesita bloquear para completar su acción?

- ◆ Si una función necesitaba bloquear para completarse asigna `EWOULDBLOCK` a `errno`
- ◆ UDP: `send()` y `sendto()` no bloquean (no hay buffer de salida)
- ◆ En TCP, `connect()` asigna `EINPROGRESS` a `errno`
- ◆ `getAddrInfo` bloquea hasta resolver el nombre

Nonblocking i/o: sockets no bloqueantes

```
numBytesRcvd = recvfrom(servSock, buffer, MAXSTRINGLENGTH, 0,  
    (struct sockaddr *) &clntAddr, &clntLen);  
if (numBytesRcvd < 0) {  
    // Only acceptable error: recvfrom() would have blocked  
    if (errno != EWOULDBLOCK)  
        ERROR("recvfrom() failed");
```

```
do {  
    res = connect(soc, (struct sockaddr *)&addr, sizeof(addr));  
    if (res < 0) {  
        if (errno == EINPROGRESS) {  
            putchar('.'); continue; }  
        ERROR  
    }  
} while ( res != 0);
```

¿ Performance ?

Nonblocking i/o: I/O asincrónico

- ◆ En vez de “polling”, que el S.O. avise cuando un “socket call” está listo.
- ◆ El programa realiza otras tareas hasta tener que manejar señal SIGIO.
 1. Usar `sigaction()` para capturar SIGIO
 2. Usar `fcntl()` para hacernos owner del socket
 3. Usar `fcntl()` para no bloquear el socket

Ver UDPEchoServer-SIGIO.c